CS-550 PA-3
Fatima Mariyam – A20527616
Design Document

Overview

The objective of this assignment is to create a hierarchical peer-to-peer (P2P) system. The system comprises two types of nodes: weak peer nodes and super-peer nodes. Weak peer nodes are linked to super-peer nodes, and they carry out tasks like hosting files, registering files with their associated super-peer, handling download requests, and updating file information. Super-peer nodes manage their connected peers, handle registration/un-registration of weak peer nodes, and facilitate file queries and propagation within the P2P network.

Project Structure

- dirs – contains subdirectories with files hosted by the peers.
- docs – documents, manuals, reports
- misc – relevant images, screenshots
- src – source code
  - com.aos
    - **Download.java** – Extends Thread and handles query receive, forward, and eventual download mechanism.
    - **FileDownloadManager.java** – download thread handler which instantiates the FileSender and responds with file as an ObjectOutputStream.
    - **FileSender.java** – initial socket creation and server listening handling.
    - **LeafNodeHandler.java** – leaf node handling
    - **Main.java** - To begin the application, we initialize integers, strings, arrays, and a static string that is passed as a parameter in the void main function of the Main class. We retrieve all arguments given on the command line and store them in variables. Then, we print which Super-peer is initiated with its specific directory and topology.

      Using the Properties function, we read the configuration file that sets up the topology between the peers. After that, we initiate the File-downloader function and the Superpeer function.

      Once all Super-peers are set, we prompt the user to input which file needs to be downloaded and store the input in a string. The topology process begins, and we retrieve details of the peer topology, port, and server port from where the information of the topology for each peer is stored using the getProperty function.

      All this information is stored in the Leafnode function, which creates a socket using this information and executes using threads. This will search the file and tell us in which Super-peer the file is present and print it in the output. The leafnode is selected from where the file is to be downloaded, and it grabs its peer and port number.

      Finally, we call the Client-server function, where the file transfer takes place.

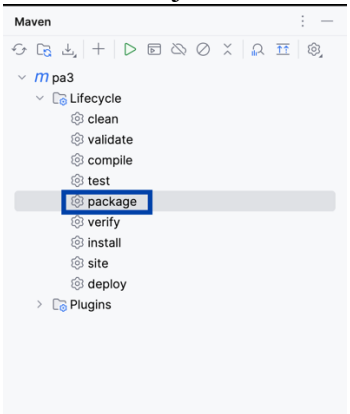
    - **MessageFormat.java** – describes the format for the query message.
    - **Superpeer.java** – Super peer mechanism handler that takes care of communicating with other peers and managing the mechanisms of TTL, message passing, message creation.

- all-to-all-topology.txt - Configuration file for all-to-all-topology topology, describing the node-to-node connections.
- tree-topology.txt – Configuration file for tree topology, describing the node-to-node connections.
- config.properties – project configuration file.
- run_peer.sh – script to start server peers using arguments like <server-number> and <topology-type>

❖ target – contains compiles class files
- pa3-1.0-SNAPSHOT.jar – executable jar

Run Instructions

* To create build jar, import the project in IDE(IntelliJ) - Build
* go to Maven configurations - lifecycle - package to generate the jar, which is created as - target/pa3-1.0-SNAPSHOT.jar



* change current directory to src

cd src

* use run_peer.sh to start super peers.

./run_peer.sh <super-peer-number> <topology-type>

where <topology-type> can be T or A2A (T: Tree topology, A2A: All-to-all topology)

Example:

./run_peer.sh 1 A2A //This starts a Super-peer 1 with all-to-all topology.

./run_peer.sh 2 T //This starts a Super-peer 2 with tree topology.

* Initialize and start super-peers as needed in different terminals.
* The neighboring weak peers are configured using the topology configuration files (tree-topology.txt and all-to-all-topology.txt)
* Available shared directories with files are at location /dirs/Peer*. Path for this can be configured in *config.properties* file.
* generate_files.sh can also be used to generate sample directories containing random number of files. (run as `./generate_file.sh`)
* Follow instructions on screen - enter file-names for querying and downloading files.

Implementation and Design Methodology

The heirarcical P2P file sharing has a static network implementation. Every peer reads the configuration file, which is called tree-topology.txt or all-to-all-topology.txt(based on selected topology). This file allows all the super peers to know which leaf-peer(or weak nodes as mentioned in the problem statement) they are connected

to and vice versa. The superpeer works as an index server for each leafpeer, so every file present in the leafpeers is registered in the superpeer.

```
****************************************
Super-peer 2
available directory ../dirs/Peer2
Topology: tree-topology.txt
****************************************
Enter the filename to download
```

When we start a super-peer, it starts Leafnodes in the background. LeafNodes presents a console to read user input for a file to search. Additionally, it starts a background thread for the Superpeer to reply to FileSearch queries. Regardless of whether the file is found or not in the Superpeer registry, the Superpeer sends the query to every neighboring Superpeer. Each Superpeer then searches its leafpeers for the file. If the file is found, the query message is sent to the path from where the request was sent earlier.

```
leaf-peer containing the file:
6

 Selecting leaf-peer: 6 for file download

file(6)_1.txt file is transferred to your private storage: .
./dirs/Peer1
File: file(6)_1.txt downloaded from leaf-peer: 6 to leaf-pee
r:1
```

To limit the query message from forwarding it to many peers, we assign the query with a value called "TTL" which gets decremented by 1 when visiting a Superpeer until it reaches 0. Every query is assigned a leafnodes ID that is unique with a sequence number. The Superpeer keeps track of the message ID for tracking where the message is receiving a leafpeer or superpeer. We use associative array pairs, which have the message ID, peer ID, and the IP address with the port number. We set the size of the associative array the same as the number of superpeers, thus flushing out the old entries whenever needed. We need to maintain this data structure for every Superpeer so that it cannot forward the message which is already sent, and we need to send it back to the reverse path for the query hit to the original sender.

The queryhit message carries the same message ID related to the same query so that we can send back the query correctly. We use a socket connection and ObjectOutputStream to send the file to the requested node.

```
New Query received from peer with peer-id : 1
Superpeer: "Search in leaf-peer completed"
Query forwarded to next neighbour: 4
Query forwarded to next neighbour: 5
```

Every Superpeer has a list of each Superpeer connected to it, and broadcasting of messages is done to all the neighbors and relayed until the "TTL" of each receiver comes down to 0. Using the reverse path, the message is sent back to the original sender.

Finally, a download request is sent to the leafpeer where the file is found, which has the same implementation as that of assignment 1.

```
New Query received from peer with peer-id : 3
File Found! queryhit: file(6)_1.txt
Superpeer: "Search in leaf-peer completed"
```